



Teoria da Computação e compiladores

Documentação Elevator

Case 2 - Elevador

Nome e RA dos Alunos: Grupo 13

- Eduardo dos Santos Berlatto, RA: 821220735
 - Guilherme Silva da Costa, RA: 82426746
- Mariana Rosendo de Freitas, RA: 821236352
- Pedro Henrique Freire de Oliveira, RA: 821224361
 - Rafael da Silva Rodrigues, RA 822165658
 - Vitor Nunes Oliveira, RA: 821224765

Sumário

Introdução.....	3
<i>Autômato com Pilha (PDA) Definição.....</i>	4
Estrutura do PDA	4
Funcionamento	4
Modos de Aceitação	4
Linguagens Reconhecidas	5
Exemplo de PDA	5
Importância e Aplicações	5
Limitações	5
Comparação com Outros Modelos	5
Definição formal do PDA	6
<i>Diagrama do Autômato de Pilha.....</i>	7
<i>Conclusão.....</i>	7
<i>Referências bibliográficas</i>	8

Introdução

O projeto apresentado foi desenvolvido em resposta ao Case 2 da disciplina de Teoria da Computação e Compiladores. A proposta envolve a modelagem de um sistema de elevador que atende o térreo e mais três andares, simulando seu comportamento mediante um Autômato de Pilha (PDA) e uma animação gráfica interativa.

A implementação foi realizada utilizando a linguagem Python com a biblioteca Pygame, proporcionando uma interface visual que simula o movimento do elevador entre os andares, a abertura e fechamento de portas, e o controle sequencial baseado em entradas de usuário.

Autômato com Pilha (PDA)

Definição

O Autômato com Pilha (do inglês, Pushdown Autômato - PDA) é um modelo computacional que estende os Autômatos Finitos ao incorporar uma pilha. Essa estrutura adicional permite que o PDA reconheça linguagens livres de contexto, que não podem ser reconhecidas por autômatos finitos simples.

Estrutura do PDA

Um PDA é definido por uma tupla (específica todos os componentes essenciais para a definição do autômato, permitindo sua descrição formal e precisa):

$$M = (Q, \Sigma, \Gamma, \Delta, q_0, Z_0, F)$$

- **Q**: conjunto finito de estados.
- **Σ (Sigma)**: alfabeto de entrada (ex: $\Sigma = \{a, b\}$).
- **Γ (Gamma)**: alfabeto da pilha (ex: $\Gamma = \{X, Z_0\}$).
- **Δ (Delta)**: função de transição $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$
- **q_0** : estado inicial.
- **Z_0** : símbolo inicial da pilha.
- **F**: conjunto de estados finais (para aceitação por estado final).

Funcionamento

O PDA lê símbolos de entrada e usa a pilha para armazenar ou remover símbolos, conforme necessário. Cada transição depende do símbolo atual da entrada (ou para transições vazias), do símbolo no topo da pilha e do estado atual. As ações possíveis incluem:

- Empilhar um símbolo (push).
- Desempilhar (remover) um símbolo (pop).
- Manter a pilha inalterada.

Modos de Aceitação

Um PDA pode aceitar uma cadeia de duas formas principais:

Por estado final: A entrada é aceita se, após consumir todos os símbolos da cadeia, o autômato estiver em um dos estados do conjunto de estados finais (ϵ).

Por pilha vazia: A entrada é aceita se, após consumir todos os símbolos da cadeia, a pilha estiver completamente vazia, independentemente do estado atual.

Apesar das diferenças operacionais entre os modos, ambos são equivalentes em termos de poder de reconhecimento, ou seja, qualquer linguagem reconhecida por um dos modos também pode ser reconhecida pelo outro.

Observação sobre o Projeto: No projeto do elevador, o modo de aceitação adotado foi por estado final. O elevador aceita a sequência de comandos quando atinge o andar selecionado, considerando o último estado como final, independente da condição da pilha.

Linguagens Reconhecidas

PDA's reconhecem exatamente as linguagens livres de contexto. Exemplos incluem:

- $L = \{a^n b^n \mid n \geq 0\}$: quantidade igual de 'a's seguidos de 'b's.
- Expressões com parênteses balanceados.

Exemplo de PDA

Para a linguagem $L = \{a^n b^n \mid n \geq 0\}$, o PDA funciona da seguinte maneira:

- Para cada símbolo 'a' lido, empilha-se um marcador (ex: X).
- Para cada símbolo 'b' lido, remove-se um marcador do topo da pilha.
- A entrada é aceita se, ao final da leitura, a pilha estiver vazia.

Esse exemplo mostra como a pilha permite ao autômato “lembrar” quantos símbolos ‘a’ foram lidos para compará-los com os ‘b’.

Importância e Aplicações

PDA's são fundamentais no estudo de compiladores, pois são usados na análise sintática de linguagens de programação. Eles também ajudam a entender os limites das linguagens que podem ser reconhecidas de forma eficiente.

Limitações

Apesar de serem mais poderosos que autômatos finitos, os Autômatos com Pilha (PDA's) ainda possuem limitações. Por exemplo, eles não conseguem reconhecer linguagens que exigem comparações entre múltiplas seções independentes da entrada, como a linguagem:

$$L = \{a^n b^n c^n \mid n \geq 0\},$$

onde é necessário garantir que a quantidade de ‘a’s, ‘b’s e ‘c’s seja igual.

Esse tipo de linguagem requer mais do que uma pilha — normalmente, uma Máquina de Turing ou um autômato com memória mais flexível seria necessário para realizar tal validação.

Comparação com Outros Modelos

- **Autômato Finito (FA):** Sem memória auxiliar, reconhece linguagens regulares.
- **Autômato com Pilha (PDA):** Com memória tipo pilha, reconhece linguagens livres de contexto.
- **Máquina de Turing (TM):** Memória infinita e bidirecional, reconhece linguagens recursivamente enumeráveis.

Definição formal do PDA:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- **Q (Estados):**

$\{q_0, q_1, q_2, q_3, q_0A, q_1A, q_2A, q_3A\}$

Estados correspondem aos pisos do elevador (q_0 a q_3), com elevador em movimento (portas fechadas), e estados auxiliares (q_0A a q_3A) representam o elevador parado com a porta aberta em cada andar.

- **Σ (Alfabeto de Entrada):**

$\{Up, Down, Open\}$

Comandos para subir, descer e abrir a porta do elevador.

- **Γ (Alfabeto da Pilha):**

$\{F\}$

Símbolo que representa a porta fechada durante a movimentação. Ele é empilhado quando o elevador se move e desempilhado quando a porta é aberta.

- **δ (Função de Transição):**

$\delta(q_0A, Up, \epsilon) \rightarrow (q_1, F)$

$\delta(q_1A, Up, \epsilon) \rightarrow (q_2, F)$

$\delta(q_2A, Up, \epsilon) \rightarrow (q_3, F)$

$\delta(q_3A, Down, F) \rightarrow (q_2, \epsilon)$

$\delta(q_2A, Down, F) \rightarrow (q_1, \epsilon)$

$\delta(q_1A, Down, F) \rightarrow (q_0, \epsilon)$

$\delta(q_0, Open, F) \rightarrow (q_0A, \epsilon)$

$\delta(q_1, Open, F) \rightarrow (q_1A, \epsilon)$

$\delta(q_2, Open, F) \rightarrow (q_2A, \epsilon)$

$\delta(q_3, Open, F) \rightarrow (q_3A, \epsilon)$

- **q_0 (Estado Inicial):**

q_0A (elevador parado no térreo com a porta aberta)

- **Z_0 (Símbolo Inicial da Pilha):**

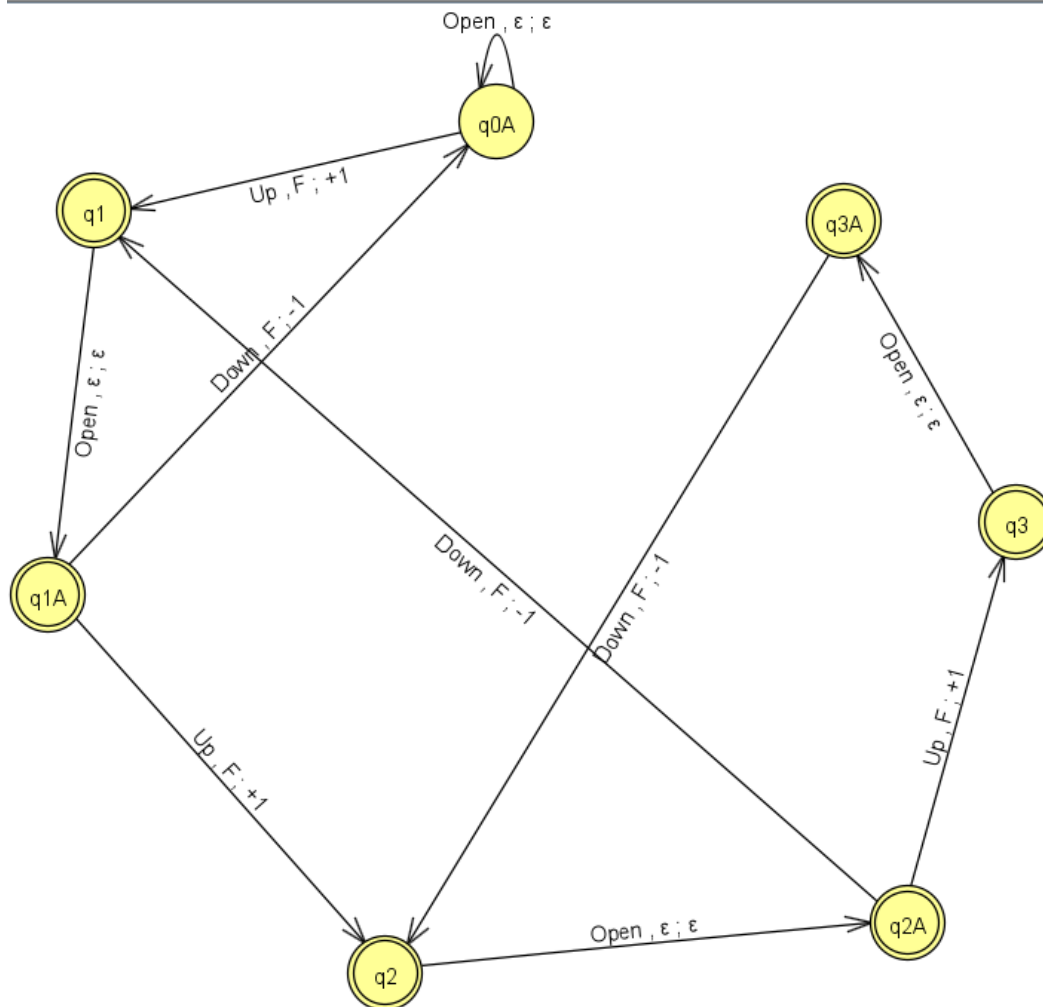
ϵ (pilha vazia no início)

- **F (Estados Finais):**

$\{q_0A, q_1A, q_2A, q_3A\}$

(O elevador pode parar com a porta aberta em qualquer andar conforme a operação do usuário.)

Diagrama do Autômato de Pilha



Conclusão

O desenvolvimento deste projeto permitiu a aplicação prática dos conceitos de Autômatos com Pilha (PDA), abordados na teoria de linguagens formais. Ao modelar um elevador utilizando transições de estados controladas por comandos de usuário (Up e Down) e gerenciamento de pilha, foi possível reforçar o entendimento sobre estruturas de memória e estados finitos.

A implementação gráfica com Python e Pygame simulou eficientemente o deslocamento do elevador e o controle de abertura/fechamento de portas, atendendo às exigências do Case 2. Esta experiência demonstrou a aplicabilidade dos conceitos de autômatos na automação de sistemas interativos e a importância de estruturas formais no design de controladores sequenciais.

Referências bibliográficas

[1] HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introdução à Teoria da Computação. 2. ed. Porto Alegre: Bookman, 2002.

[2] SIPSER, Michael. Introdução à Teoria da Computação. 2. ed. Cengage Learning, 2010.